

Rapport Final du projet PRONTO n° 83

IMT Atlantique — Campus de Brest

Présenté par

Elias HASNAOU	Chef de projet
Christian TRANDAFIR	
Ayoub DAOUDI	
Sami GUESSOUM	
Mohamed CHAABANE	
Salomé GOULIN	

Reconnaissance Automatique de Cartes Pokémon

Identification visuelle par intelligence artificielle

Rendu le 26 mai 2026 à l'encadrant technique :

Bastien Padeloup

Résumé

Ce rapport final présente l'aboutissement du projet PRONTO 83, dont l'objectif est de concevoir un système de reconnaissance automatique de cartes Pokémon à partir d'images en conditions réelles. Le système doit identifier, parmi un catalogue de plus de 20 000 variantes, la carte présente sur une photographie quelconque, en moins de cinq secondes et avec un taux de rappel **Recall@1** supérieur à 80 %.

La solution développée repose sur un pipeline modulaire en six étapes : détection et redressement géométrique de la carte (P1), extraction d'une empreinte visuelle discriminante (P2), recherche par similarité dans la galerie de référence (P3), extraction de texte par OCR pour validation complémentaire (P4), exposition via une API et une interface web (P5), et visualisation des décisions par cartes d'attention (P6).

Les principales contributions scientifiques et techniques portent sur deux modules :

- (i) la reconstruction du jeu de données de détection à partir d'images terrain annotées automatiquement avec SAM3.1, suivie de l'entraînement d'un YOLO26n-Seg qui surpasse de plus de 10 points le précédent modèle tout en étant 8,7 fois plus petit ;
- (ii) la construction d'un *teacher* d'empreintes visuelles DINOv3-Large entraîné par une perte contrastive supervisée multi-positifs, faisant passer le **Recall@1** sur les photos Vinted de 79,29 % à 98,59 %, puis sa distillation vers des modèles 10 fois plus légers sans perte de performance.

Le rapport revient également sur la gestion du projet, les ajustements de planification, et le retour d'expérience individuel et collectif sur la dynamique d'équipe.

Mots-clés : vision par ordinateur, segmentation, YOLO, transformers, apprentissage contrastif, distillation, recherche par similarité, FAISS, cartes Pokémon.

Table des matières

Résumé	i
1 Introduction	1
1.1 Contexte	1
1.2 Problématique	1
1.3 Objectifs	1
1.4 Annonce du plan	2
2 Rappel du cahier des charges fonctionnel	2
2.1 Contexte d'utilisation	2
2.2 Expression fonctionnelle	2
2.3 Critères de validation	3
3 Conception et architecture du système	3
3.1 Vue d'ensemble du pipeline	3
3.2 Choix technologiques et justifications	4
3.3 Décomposition en lots de travail (WBS)	5
3.4 Évolution de l'architecture en cours de projet	5
4 Développement des composants	6
4.1 Module P1 – Détection et redressement	6
4.2 Module P2 – Empreinte visuelle	10
4.3 Module P3 – Recherche par similarité	16
4.4 Module P4 – OCR et métadonnées	17
4.5 Module P5 – API et interface web	19
4.6 Module P6 – Explicabilité	20
5 Intégration et validation du système	21
5.1 Stratégie d'intégration	21
5.2 Scénarios de validation	22
5.3 Benchmark global sur photos terrain	22
6 Clôture du projet	22
6.1 Tableau des livrables	22
6.2 Validation : scénarios de démonstration et expériences	23
6.3 Axes d'amélioration du système	23
7 Retour d'expérience sur la gestion de projet	24
7.1 Planification temporelle	24
7.2 Analyse des écarts	25
7.3 Gestion des risques	26
7.4 Recommandations pour un projet similaire	27
8 Retour d'expérience sur le travail en équipe	27
8.1 Organisation et fonctionnement	27

8.2	Ce qui a bien fonctionné	27
8.3	Difficultés rencontrées et résolutions	28
8.4	Plan d’actions suite à l’atelier sur la dynamique d’équipe	28
8.5	Recommandations pour la suite de la formation	28
9	Conclusion et perspectives	29
9.1	Bilan du projet	29
9.2	Apports scientifiques et techniques	29
9.3	Perspectives à court et moyen terme	29
	Références	30
A	Retours d’expérience individuels	30
A.1	Christian Trandafir	30
A.2	Elias Hasnaou	32
A.3	Mohamed Chaabane	33
A.4	Ayoub Daoudi	34
A.5	Salomé Goulin	36
A.6	Sami Guessoum	37

1 Introduction

1.1 Contexte

Le marché des cartes à collectionner Pokémon connaît un essor considérable, avec un catalogue de plus de 20 000 variantes. Collectionneurs, revendeurs et passionnés ont besoin d'identifier rapidement et précisément chaque carte, que ce soit pour estimer sa valeur, vérifier son authenticité ou gérer une collection. Cette identification, aujourd'hui manuelle, est chronophage, sujette à erreur, et requiert une expertise importante.

Les plateformes de revente en ligne (Vinted, eBay, CardMarket) génèrent un volume croissant de photos de cartes, souvent prises dans des conditions dégradées : angles variés, reflets dus aux cartes holographiques, arrière-plans encombrés et faible résolution. À ce jour, aucun outil grand public ne permet une identification automatique fiable dans ces conditions réelles. Les méthodes actuelles sont essentiellement fondées sur l'OCR ce qui présente 2 défauts majeurs : le texte disparaît bien avant l'empreinte visuelle globale de l'image ce qui invalide cette solution si la photo est floue ou présente des reflets ; le texte seul ne permet pas de différencier deux cartes avec un texte identique, par exemple l'édition 1 *vs* l'édition 2.

1.2 Problématique

Le projet répond donc à la question suivante :

Comment identifier automatiquement et de manière fiable une carte Pokémon parmi plus de 20 000 variantes à partir d'une simple photographie prise en conditions réelles, en combinant techniques de détection d'objets, d'apprentissage de représentations visuelles et de recherche par similarité ?

1.3 Objectifs

Les objectifs scientifiques et techniques du projet, fixés lors de la phase de cadrage, sont les suivants :

- O1. Détection robuste** : localiser et recadrer automatiquement une carte Pokémon dans une image quelconque, avec une précision supérieure à 90 %.
- O2. Identification précise** : identifier la carte parmi 20 000 variantes avec un Recall@1 supérieur à 80 % sur des images réelles.
- O3. Temps de réponse acceptable** : assurer un temps de traitement inférieur à 5 secondes.
- O4. Interface démontrable** : fournir une application web permettant d'uploader une image et d'obtenir le résultat d'identification.
- O5. Explicabilité** : offrir des visualisations montrant les zones de l'image utilisées par le modèle pour sa prédiction.

1.4 Annonce du plan

Le présent rapport rend compte de l'état final du projet selon ses axes scientifique, technique et organisationnel. La section 2 rappelle le cahier des charges fonctionnel issu de la phase de cadrage. La section 3 présente la conception générale du système et les choix d'architecture retenus. La section 4 détaille le développement de chacun des six modules, en justifiant les choix techniques au regard des résultats obtenus. La section 5 décrit l'intégration et la validation du système complet. La section 6 clôt la partie scientifique par un tableau des livrables, des scénarios de validation et les axes d'amélioration identifiés. Les sections 7 et 8 sont consacrées au retour d'expérience sur la gestion de projet et le travail en équipe. Enfin, la section 9 conclut sur les perspectives ouvertes par le projet.

2 Rappel du cahier des charges fonctionnel

2.1 Contexte d'utilisation

Le système est destiné à un utilisateur (collectionneur ou revendeur) souhaitant identifier rapidement une carte Pokémon. L'utilisateur prend une photo de la carte avec son téléphone ou utilise un *screenshot* (par exemple depuis une annonce Vinted), l'envoie via l'interface web et obtient en retour l'identification de la (ou des) carte(s) assortie(s) d'un score de confiance. Les conditions d'utilisation typiques incluent des photos prises au smartphone de qualité variable, des cartes inclinées ou partiellement masquées, des fonds encombrés, des reflets holographiques, ainsi que des *screenshots* de plateformes de vente avec éléments parasites (prix, icônes, etc.), plusieurs cartes sur l'image.

2.2 Expression fonctionnelle

Les huit fonctions principales du système sont rappelées dans le tableau 1.

ID	Fonction	Description	Priorité
F1	Détection de carte	Localiser une carte dans l'image, quelle que soit l'orientation ou le fond.	Essentielle
F2	Redressement	Produire une vue frontale normalisée de la carte détectée.	Essentielle
F3	Extraction de <i>features</i>	Encoder l'image redressée en un vecteur discriminant.	Essentielle
F4	Recherche par similarité	Retrouver les cartes candidates les plus proches dans la base de référence.	Essentielle
F5	Validation géométrique	Affiner le classement par matching de points locaux.	Importante
F6	Extraction OCR	Extraire le texte (nom, numéro) pour valider ou compléter l'identification.	Optionnelle
F7	Interface web	Fournir une interface intuitive d'upload et d'affichage des résultats.	Essentielle
F8	Explicabilité	Visualiser les zones contribuant à la décision du modèle.	Importante

TABLE 1 – Expression fonctionnelle du système.

2.3 Critères de validation

Les critères de validation, présentés dans le tableau 2, servent de référence pour évaluer l'atteinte des objectifs.

ID	Critère de validation	Seuil d'acceptation
V1	Précision de détection (P1)	> 90 % sur un jeu de test de 100 images variées
V2	Recall@1 identification (P2+P3)	> 80 % sur images réelles
V3	Recall@5 identification (P2+P3)	> 95 % sur images réelles
V4	Temps de réponse end-to-end	< 5 secondes
V5	Robustesse aux reflets holo	Dégradation < 15 % par rapport aux cartes non-holo
V6	Démonstration fonctionnelle	Pipeline complet opérationnel à la soutenance

TABLE 2 – Critères de validation du besoin.

3 Conception et architecture du système

3.1 Vue d'ensemble du pipeline

Le système est structuré autour d'un pipeline séquentiel composé de six modules interconnectés, chacun traitant une étape spécifique du processus d'identification.

La figure 1 présente le schéma bloc global, de la photographie d'entrée jusqu'à l'identification finale et son explication.

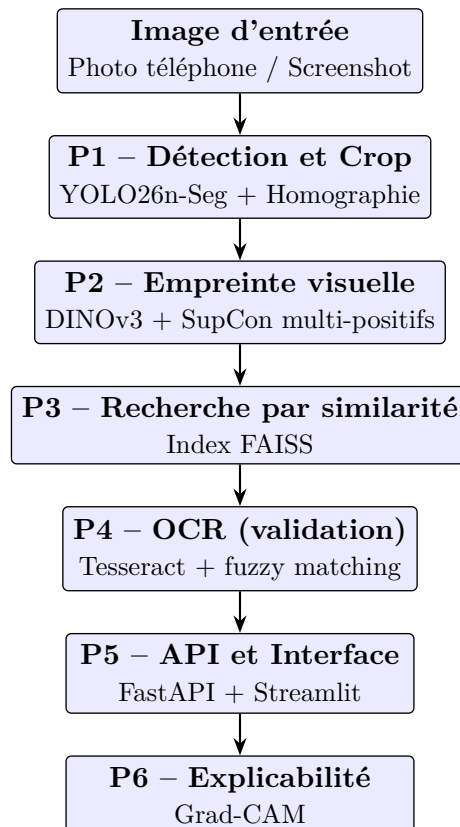


FIGURE 1 – Vue d'ensemble du pipeline modulaire du système.

3.2 Choix technologiques et justifications

L'ensemble des choix technologiques a été guidé par trois exigences : la performance en conditions réelles, la légèreté à l'inférence, et la possibilité de déployer une démonstration interactive. Le tableau 3 synthétise les choix retenus pour chaque module ainsi que leur justification principale.

Module	Technologie retenue	Justification
P1	YOLO26n-Seg (<i>tiny</i>) + OpenCV (homographie)	Segmentation par instance native, modèle léger (2,69 M de paramètres) compatible CPU, écosystème mature.
P2	DINOv3-Large fine-tuné par SupCon multi-positifs, puis distillé sur ViT-S+ et ConvNeXt-Small	Backbone auto-supervisé de l'état de l'art, capable de capturer des détails fins. Distillation pour le déploiement temps réel.
P3	FAISS (<i>FlatIP</i>)	Recherche par produit scalaire rapide, scalable à plusieurs dizaines de milliers de vecteurs, naturel pour des embeddings normalisés L2.
P4	Tesseract + segmentation par zones	Pipeline simple et modulaire pour valider la faisabilité, sans introduction de dépendance lourde.
P5	FastAPI + Streamlit	API performante avec validation de schéma automatique ; interface web rapide à prototyper pour la démonstration.
P6	Grad-CAM	Méthode d'explicabilité de référence pour les réseaux convolutifs et transformers, peu coûteuse à intégrer.

TABLE 3 – Synthèse des choix technologiques par module.

3.3 Décomposition en lots de travail (WBS)

La figure 2 présente la décomposition du projet en cinq lots principaux (niveau 2), conformément à la structure établie lors du cadrage.

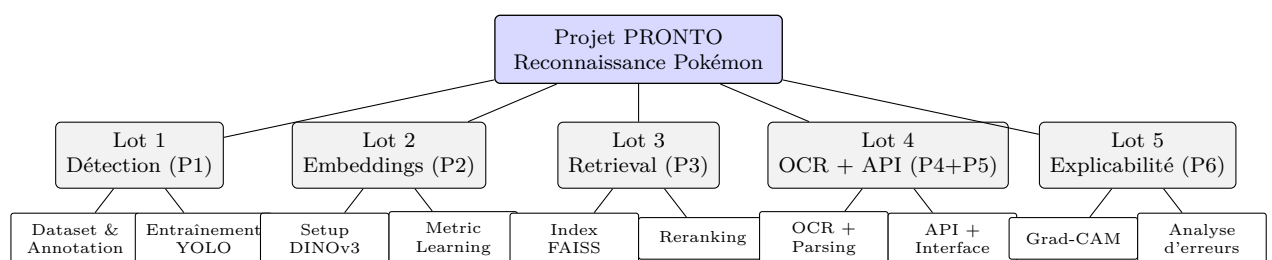


FIGURE 2 – Structure de découpe du projet (WBS, niveau 2).

3.4 Évolution de l'architecture en cours de projet

L'architecture initiale, telle que définie dans le document de cadrage, a connu trois ajustements majeurs au cours du projet, motivés par les observations expérimentales :

- **Abandon de ArcFace au profit de SimCLR (puis SupCon) pour P2.** Les résultats comparatifs à 224×224 ont montré une supériorité nette de

l'apprentissage contrastif, mieux adapté à un contexte *one-shot* où une seule image de référence est disponible par carte (cf. section 4.2).

- **Passage de la résolution 224 à 518 pour P2.** La nature *fine-grain* du problème, où deux cartes se distinguent par des détails de petite taille (numéro, icône d'extension), a justifié l'augmentation de résolution, qui a fait passer le `Recall@1` sur la galerie propre de 70 % à 98 %.
- **Abandon de LightGlue pour P3.** Le `Recall@1` obtenu par P2 étant déjà très élevé après les améliorations successives, le *reranking* géométrique est devenu inutile, d'autant plus que l'ordre des candidats au-delà du rang 1 n'est pas suffisamment fiable pour bénéficier d'un raffinement local.

4 Développement des composants

4.1 Module P1 – Détection et redressement

4.1.1 Motivation et enjeux

Ce premier module est crucial pour le projet. La qualité du *crop* conditionne l'ensemble des traitements en aval : si la carte est mal détectée, les modules suivants n'ont aucune chance de fonctionner correctement. Vu l'importance de cette étape, une grande partie des efforts du projet a été consacrée à son développement et à son perfectionnement, qui s'est étalé sur les deux phases de production.

4.1.2 Première itération : YOLO26m-Seg sur dataset synthétique

Sortie du modèle. À partir d'une image en entrée, le modèle YOLO26m-Seg produit, pour chaque carte détectée, un masque de segmentation (contour précis), une boîte englobante, un score de confiance et une classe unique : `pokemon_card`.

Pipeline géométrique de redressement. Une fois la carte détectée via son masque de segmentation, un pipeline géométrique permet de produire une vue frontale normalisée. Ce pipeline est intégré dans la fonction `detect_and_crop` et repose sur les étapes suivantes :

- **Extraction et simplification du contour :** récupération du polygone brut décrivant le contour, application d'une enveloppe convexe, puis approximation polygonale itérative pour obtenir exactement quatre sommets ; en cas d'échec, repli sur un rectangle orienté minimal.
- **Identification et ordonnancement des coins :** les quatre sommets sont ordonnés selon une convention fixe (Haut-Gauche, Haut-Droit, Bas-Droit, Bas-Gauche) via un critère sur la somme et la différence des coordonnées.
- **Normalisation de l'orientation :** détection portrait / paysage en comparant les dimensions, avec rotation post-transformation si nécessaire.

— **Transformation perspective** : estimation d’une homographie par

`cv2.getPerspectiveTransform`,

puis application via `cv2.warpPerspective` vers un format cible 512×715 px correspondant aux proportions standard d’une carte.

Constitution du dataset. Le dataset initial a été annoté via Roboflow, en segmentation polygonale. Les images sont des compositions de cartes Pokémon superposées sur différents fonds afin de couvrir une large variété de conditions visuelles. Il comprend 725 images d’entraînement et 71 images de validation, soit 796 images au total.

Pour augmenter la robustesse sans collecte supplémentaire, un script `augment_rotations.py` multiplie le dataset par un facteur 4 en ajoutant des rotations de 90° , 180° et 270° . Les coordonnées des polygones d’annotation sont mises à jour par les formules suivantes (coordonnées normalisées) :

$$\begin{aligned} \text{rotation } 90^\circ : \quad (x', y') &= (1 - y, x) \\ \text{rotation } 180^\circ : \quad (x', y') &= (1 - x, 1 - y) \\ \text{rotation } 270^\circ : \quad (x', y') &= (y, 1 - x) \end{aligned}$$

Après augmentation, le dataset contient 2 900 images d’entraînement et 284 images de validation, soit 3 184 images au total.

Entraînement. L’entraînement a été réalisé en deux phases sur une GTX 1660 Ti (6 Go VRAM), ce qui impose la désactivation de l’AMP en raison d’instabilités numériques. La phase 1 (60 epochs, *batch* adaptatif $4 \rightarrow 2$, learning rate $10^{-2} \rightarrow 10^{-4}$) entraîne le modèle pré-entraîné sur le dataset initial. La phase 2 fine-tune le meilleur `best.pt` sur le dataset augmenté en gelant le *backbone* (10 premières couches), avec un learning rate réduit à 5×10^{-4} et une augmentation *mosaic* abaissée à 0,4 afin de limiter le *catastrophic forgetting*.

Résultats de la première itération. Les indicateurs de performance sont présentés dans le tableau 4.

Métrique	Phase 1	Phase 2	Évolution
Box Precision	0,941	0,954	+1,3 %
Box Recall	0,948	0,933	−1,5 %
Box mAP50	0,973	0,962	−1,1 %
Box mAP50-95	0,891	0,884	−0,7 %
Mask mAP50-95	0,823	0,826	+0,3 %

TABLE 4 – Performances du modèle YOLO26m-Seg aux phases 1 et 2.

L'objectif fixé d'une précision supérieure à 90 % est largement atteint, avec une précision de 95,4 % et une mAP50 supérieure à 96 %. La légère baisse de *recall* entre les phases 1 et 2 est compensée par une plus grande robustesse aux variations d'orientation. Sur la GTX 1660 Ti, le temps d'inférence moyen est de 29,5 ms, et le pipeline complet (détection + redressement) reste sous les 50 ms par image, ce qui le rend compatible avec une utilisation en temps réel.

4.1.3 Seconde itération : reconstruction du dataset via SAM3.1 et passage à YOLO26n-Seg

Lors de la revue de mi-parcours, deux limites du module P1 ont été identifiées : le dataset reposait majoritairement sur des images composées artificiellement, peu représentatives des conditions réelles ; et le modèle déployé (YOLO26m-Seg) était relativement lourd au regard d'une tâche à classe unique. Cette seconde itération poursuit un double objectif : reconstruire un dataset à partir d'images réelles à coût d'annotation maîtrisé, et entraîner un modèle plus léger.



Image artificielle



Image réelle

FIGURE 3 – Comparaison entre les images artificielles utilisées dans le premier dataset YOLO et les images réelles collectées ensuite. Cette différence illustre la nécessité de reconstruire un dataset plus représentatif des conditions terrain avant l'annotation automatique par SAM3.1.

Pipeline d'annotation SAM3.1 → YOLO segmentation. L'idée centrale est de remplacer l'annotation manuelle par une génération automatique de masques de segmentation suivie d'une phase de nettoyage léger. Le script principal

```
annotate_sam3_to_yolo.py
```

enchaine : (1) chargement de l'image en RGB ; (2) application de SAM3 ou SAM3.1 avec un *prompt* textuel (*card, trading card...*) ; (3) filtrage des masques selon

un score minimal et une aire minimale ; (4) NMS sur masques ; (5) conversion de chaque masque conservé en contour polygonal puis simplification ; (6) normalisation et écriture au format YOLO segmentation.

Les seuils retenus sont `conf_thres=0.35`, `mask_iou_thres=0.85`, `min_mask_area=600` px et `val_ratio=0.10`. Le *split* train / validation est effectué par groupes d’images originales pour éviter qu’une image et ses rotations ne se retrouvent dans deux *splits* différents, ce qui surestimerait artificiellement les métriques.

Comparatif SAM3 vs SAM3.1. Un premier essai avec SAM3 et un *prompt* générique offre une couverture très limitée : 77,2 % des images ne reçoivent aucune annotation, ce qui rend l’apprentissage instable. Le passage à SAM3.1 avec un *prompt* mieux ciblé améliore très nettement la qualité (cf. tableau 5). Les scores médians des masques produits par SAM3.1 atteignent 0,898, avec un 90^e percentile à 0,945.

Indicateur	SAM3.1 brut	SAM3.1 nettoyé
Images traitées	1 264	1 231
Images non vides	1 231	1 231
Images sans label	33	0
Polygones produits	4 455	4 455

TABLE 5 – Statistiques du dataset SAM3.1 avant et après nettoyage.

Dataset final combiné. Le dataset SAM3.1 nettoyé est ensuite fusionné avec le dataset artificiel précédent et leurs rotations, pour aboutir à un dataset combiné final de 4 600 images (4 036 train / 412 val / 152 test) totalisant 13 461 instances annotées.

Comparaison directe avec l’ancien YOLO m. Les deux modèles sont évalués sur un même *split* de test (152 images, 596 instances), à `imgsz=640` et avec une classe unique. Les résultats, résumés dans le tableau 6, montrent que le nouveau YOLO *nano* surpasse l’ancien YOLO m sur l’ensemble des métriques tout en étant $\approx 8,7\times$ plus petit en paramètres et $\approx 7,9\times$ plus rapide à l’inférence CPU.

Modèle	Params	GFLOPs	Box mAP50	Box mAP50-95	Mask mAP50-95	CPU/img
YOLO26m-Seg (ancien)	23,51 M	121,2	0,9546	0,7866	0,7504	622 ms
YOLO26n-Seg (nouveau)	2,69 M	9,0	0,9909	0,8730	0,8512	78 ms

TABLE 6 – Comparaison de l’ancien et du nouveau YOLO sur le *split* test commun.

L’écart le plus significatif concerne la Mask mAP50-95 (0,7504 $\rightarrow$ 0,8512) : l’amélioration ne se limite pas à une meilleure détection globale, elle concerne aussi la finesse des contours segmentés. Au-delà des métriques, le gain principal s’observe en conditions d’usage : le pipeline est désormais exposé à des photos réelles, et

les overlays de validation montrent une nette amélioration sur les cartes inclinées, partiellement masquées ou placées sur des arrière-plans complexes.

Interfaces et outils de démonstration. Deux outils ont été développés autour du module P1 :

- `demo_crop_align.py` : interface Gradio permettant de charger une image, d'ajuster le seuil de confiance et de visualiser les résultats du pipeline (détection et cartes redressées), accompagnée d'un panneau de *benchmark* en temps réel ;
- `webcam_live_crop.py` : application temps réel sur flux vidéo, affichant simultanément les polygones de segmentation et les cartes redressées, utilisée pour tester la robustesse en conditions réelles.



Image d'entrée



Carte détournée et redressée

FIGURE 4 – Exemple de sortie du module P1. Le modèle YOLO segmente la carte dans une image réelle contenant du fond et des obstacles, puis le pipeline géométrique applique une correction de perspective afin d'obtenir une carte recadrée et normalisée.

4.2 Module P2 – Empreinte visuelle

4.2.1 Motivation et enjeux

Une fois la carte correctement isolée et redressée par le module P1, l'objectif du module P2 est d'en extraire un vecteur de caractéristiques (*embedding*) robuste et

discriminant. Ce module est le cœur du système de reconnaissance : c’est sur la qualité de cet espace latent que reposent l’ensemble des performances finales.

Le module P2 a connu quatre évolutions successives au cours du projet, chacune motivée par une analyse fine des résultats obtenus.

4.2.2 Étape 1 – Choix du paradigme : SimCLR *vs* ArcFace

Deux grands paradigmes ont été évalués et comparés sur le modèle de fondation DINOv3 en résolution 224×224 :

- une approche contrastive (**SimCLR**) [1], visant à rapprocher les représentations d’images similaires tout en éloignant les images dissemblables ;
- une approche par classification métrique (**ArcFace**) [2], séparant strictement les classes sur une hypersphère latente via une marge angulaire.

Protocoles d’entraînement. Pour SimCLR, la perte contrastive NT-Xent (*Normalized Temperature-scaled Cross Entropy*) est minimisée avec un paramètre de température $\tau = 0,07$. Pour ArcFace, la tête de classification est configurée avec une échelle $s = 30,0$ et une marge angulaire $m = 0,50$, optimisée par AdamW avec un *warmup* de 5 % des itérations suivi d’une décroissance cosinus.

Résultats comparatifs. Les modèles sont évalués via un index FAISS *FlatIP* sur notre ensemble de test (tableau 7).

Modèle	R@1	R@5	R@10	MRR@10	ms/img	img/s
simclr_224	0,7000	0,8444	0,8889	0,7607	188,48	5,31
arcface_224	0,4520	0,5810	0,6350	0,5105	218,86	4,57

TABLE 7 – Comparaison SimCLR / ArcFace à résolution 224×224 .

Analyse. La supériorité de SimCLR (R@1 de 70 % contre 45 % pour ArcFace) s’explique par plusieurs facteurs structurels :

1. **Généralisation *vs* séparation rigide.** L’apprentissage contrastif s’affranchit du concept de classes figées et apprend une similarité relative continue, ce qui est idéal pour un système de *retrieval* où le nombre de cartes est immense et où plusieurs cartes partagent les mêmes *artworks* réédités.
2. **Contrainte de la marge angulaire.** Forcer la séparation absolue de milliers de classes sur une hypersphère contrainte dégrade le pouvoir de discrimination du modèle de fondation au profit d’une séparation trop agressive.
3. **Rareté des données intra-classe.** Notre configuration s’apparente à un problème *one-shot* : une seule image de référence nette par carte, les variations étant induites par augmentations. ArcFace nécessite à l’inverse une grande diversité d’images réelles par classe pour modéliser correctement la variance intra-classe.

4. **Efficacité computationnelle.** Le temps d'inférence de SimCLR (188 ms/img) est inférieur à celui d'ArcFace (219 ms/img).

Décision retenue : ArcFace est définitivement abandonné au profit de l'apprentissage contrastif pour la suite des expérimentations.

4.2.3 Étape 2 – Passage à la résolution 518×518

Motivation. À 224×224, les détails fins (numéro de carte, icônes d'extension) sont trop compressés. Or notre problème est *fine-grain* : deux cartes peuvent être visuellement presque identiques au premier regard. Le passage à 518×518 permet de conserver davantage d'information locale utile à l'empreinte visuelle.

Résultats. L'impact est spectaculaire (tableau 8) : le R@1 passe de 70 % à 98,23 % sur la galerie propre, et le R@5 atteint 99,84 %.

Métrique	SimCLR 224	SimCLR 518
R@1	70,00 %	98,23 %
R@5	84,40 %	99,84 %
Top-1 error	30,00 %	1,77 %
Marge moyenne	–	0,3135
Hard subset R@1	–	92,93 %

TABLE 8 – Impact du passage de la résolution 224 à 518 sur les performances de l'empreinte visuelle.

Cette résolution devient la nouvelle base pour les itérations suivantes.

4.2.4 Étape 3 – Cycle de *hard negative mining*

Idée. Le *warm-up* SimCLR à 518×518 règle déjà les cas faciles. Pour améliorer la séparation des cas ambigus (cartes très proches visuellement mais incorrectes), un protocole itératif de *hard negative mining* est appliqué : extraction des voisins les plus difficiles dans l'espace latent, puis ré-entraînement sur ces exemples.

Résultats. Le tableau 9 présente l'évolution des métriques au fil des cycles. Le signal clé est la progression du *hard subset* : 92,93 % → 95,39 %. La marge moyenne augmente continûment, signe d'un espace d'embeddings plus propre. Le meilleur checkpoint retenu est le **cycle 4**.

Phase	R@1	Hard R@1	Marge	Top-1 err.
Warm-up	98,23 %	92,93 %	0,3135	1,77 %
Cycle 1	98,60 %	94,41 %	0,3420	1,40 %
Cycle 2	98,58 %	94,31 %	0,3469	1,43 %
Cycle 3	98,69 %	94,75 %	0,3614	1,32 %
Cycle 4	98,85 %	95,39 %	0,3814	1,16 %

TABLE 9 – Évolution des performances au fil des cycles de *hard negative mining*.

4.2.5 Étape 4 – *Teacher* multi-positifs (SupCon) sur dataset terrain

Motivation. Sur le benchmark dataset3 (galerie propre), le *teacher* cycle 4 affichait des performances excellentes. Mais sur le benchmark Vinted-vers-galerie, qui correspond à l’usage réel, il plafonnait à 79,29 % de R@1 avec un rang moyen autour de 30. L’espace latent était bien structuré sur la galerie propre, mais transférait mal vers les photos réelles.

Construction du dataset terrain et choix de la loss. Un nouveau dataset terrain a donc été construit, associant à chaque carte entre 3 et 10 images réelles (galerie, photos Vinted, photos floues, inclinées, etc.). Plusieurs images correspondent désormais à une même carte : il devient nécessaire d’adapter la loss pour intégrer plusieurs positifs. La NT-Xent classique est remplacée par une *supervised contrastive loss* [5] multi-positifs. Pour chaque exemple i , on définit l’ensemble $\mathcal{P}(i)$ des autres exemples du batch ayant le même identifiant de carte, et la loss s’écrit :

$$\mathcal{L}_i = -\frac{1}{|\mathcal{P}(i)|} \sum_{p \in \mathcal{P}(i)} \log \frac{\exp(\text{sim}(z_i, z_p)/\tau)}{\sum_{a \neq i} \exp(\text{sim}(z_i, z_a)/\tau)} \quad (1)$$

avec $\tau = 0,07$. La loss totale du *teacher* ajoute une perte auxiliaire d’édition (cross-entropy binaire masquée) :

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{multi-pos SupCon}} + 0,25 \cdot \mathcal{L}_{\text{édition masquée}} \quad (2)$$



Carte édition 1



Carte édition 2



FIGURE 5 – Exemple de différence fine entre deux variantes très proches d’une même carte. La perte auxiliaire d’édition vise à inciter le modèle à tenir compte de détails discriminants comme le logo d’édition 1, qui permet de distinguer deux cartes visuellement presque identiques.

Le *sampler* est équilibré par carte : 32 cartes par batch local avec 2 exemples par carte, en associant si possible une image galerie et une photo Vinted de la même carte.

Procédure d’entraînement. Le run multi-positifs fine-tune le meilleur *teacher* SimCLR du cycle 4 (1 500 *steps*). Les principaux hyperparamètres sont rappelés dans le tableau 10.

Paramètre	Valeur
<i>Backbone</i>	facebook/dinov3-vitl16-pretrain-lvd1689m
Résolution	518
Dimension d’embedding	512 (normalisé L2)
<i>Steps</i> planifiés	1 400
Batch par GPU	64
<i>samples_per_card</i>	2
LR initial / minimal	$8 \cdot 10^{-5}$ / 10^{-6}
<i>Weight decay</i>	0,03
Précision	bf16

TABLE 10 – Hyperparamètres du *teacher* multi-positifs.

Résultats : galerie propre. Les performances restent quasiment inchangées sur dataset3 (tableau 11), ce qui constitue un prérequis avant d’évaluer le gain attendu sur Vinted.

Modèle	R@1	R@5	Hard R@1	Marge moy.
<i>Teacher</i> initial (cycle 4)	98,56 %	99,90 %	94,24 %	0,3071
SupCon multi-pos (step 1 200)	98,58 %	99,99 %	94,33 %	0,1793

TABLE 11 – Évaluation sur dataset3 – galerie propre (18 904 requêtes).

Résultats : Vinted-vers-galerie. C’est sur ce benchmark que l’expérience prend tout son sens (tableau 12). Le gain absolu en R@1 est de **+19,29 points**, et le rang moyen passe de 30,73 à 1,015. Le sous-groupe **hard_expected_in_top10**, qui regroupe les cas les plus difficiles, passe de 65,5 % à 95,0 % de R@1.

Modèle	R@1	R@5	Rang moyen	Marge moy.
<i>Teacher</i> initial (cycle 4)	79,29 %	89,01 %	30,73	0,1092
<i>Teacher</i> multi-positifs (step 1 200)	98,59 %	99,99 %	1,015	0,1548

TABLE 12 – Évaluation Vinted-vers-galerie – cas d’usage réel.

Interprétation. Le principal gain ne se limite pas à un R@1 plus élevé : il se situe surtout dans la cohérence de l’espace latent entre images galerie et photos Vinted. La loss multi-positifs rend le modèle plus robuste aux reflets et aux flous en regroupant toutes les vues d’une même carte. En conséquence, lors de l’inférence, le voisinage d’une photo Vinted dans l’espace latent contient presque toujours la bonne carte de référence.

4.2.6 Étape 5 – Distillation vers des modèles légers

Motivation. Le *teacher* DINOv3-L atteint d’excellentes performances mais reste lourd (~ 300 M de paramètres, résolution 518). L’objectif de la distillation est de transférer ses connaissances vers des étudiants plus légers, tout en conservant les performances. Deux familles d’étudiants sont étudiées :

- un étudiant Vision Transformer : `facebook/dinov3-vits16plus-pretrain-lvd1689m` (**ViT-S+**, ~ 29 M de paramètres) ;
- un étudiant convolutionnel : `facebook/convnext-small-224` (**ConvNeXt-Small**, ~ 50 M de paramètres).

Loss de distillation. La loss combine quatre termes complémentaires :

$$\mathcal{L} = 0,50 \cdot \mathcal{L}_{\text{metric}} + 0,35 \cdot \mathcal{L}_{\text{relation}} + 0,15 \cdot \mathcal{L}_{\text{embedding}} + 0,10 \cdot \mathcal{L}_{\text{édition}} \quad (3)$$

où $\mathcal{L}_{\text{metric}}$ est la SupCon multi-positifs sur les vues étudiant, $\mathcal{L}_{\text{relation}}$ est une KL-divergence entre les distributions de similarités intra-batch du *teacher* et de l’étudiant, $\mathcal{L}_{\text{embedding}}$ est un alignement cosinus direct, et $\mathcal{L}_{\text{édition}}$ est la cross-entropy binaire masquée pour la tête auxiliaire. Le *teacher* reste gelé durant toute la distillation.

Résultats. Les deux étudiants restent à moins de 0,1 point de R@1 du *teacher*, avec un R@5 presque saturé (tableau 13). Le ViT-S+ dépasse même légèrement le *teacher* (+0,09 point) avec une marge moyenne plus élevée (0,1994 contre 0,1548), signe d’un espace d’embeddings plus séparé.

Modèle (checkpoint)	R@1	R@5	Rang moyen	Marge moy.
<i>Teacher</i> DINOv3-L (step 1 200)	98,586 %	99,994 %	1,015	0,1548
ViT-S+ distillé (step 3 000)	98,680 %	99,991 %	1,014	0,1994
ConvNeXt-Small distillé (step 2 500)	98,497 %	99,986 %	1,017	0,1876

TABLE 13 – Performances Vinted-vers-galerie après distillation (34 859 requêtes, galerie de 18 904 cartes).

Impact sur le temps d’inférence. Le tableau 14 estime le coût relatif d’inférence par rapport au *teacher*. La combinaison *moins de paramètres + résolution réduite* rend la distillation pertinente pour le déploiement : à qualité quasiment préservée, on gagne près d’un ordre de grandeur sur le temps d’inférence.

Modèle	Params	Résolution	# tokens	Coût relatif
<i>Teacher</i> DINOv3-L	~300 M	518	~1 024	1× (réf.)
ViT-S+ distillé	~29 M	384	~576	~0,06–0,10×
ViT-S+ distillé (à 518)	~29 M	518	~1 024	~0,10–0,15×
ConvNeXt-Small distillé	~50 M	384	–	~0,10–0,15×

TABLE 14 – Estimation du coût relatif d’inférence par rapport au *teacher*.

4.2.7 Bilan du module P2

Les évolutions successives du module P2 ont permis de faire passer le **Recall@1** sur le cas d’usage réel (Vinted-vers-galerie) de **79,29 % à 98,68 %**, soit un gain absolu de près de 20 points, tout en divisant par 10 environ le nombre de paramètres du modèle déployé. Le **Recall@5** est presque saturé : les erreurs restantes sont principalement des erreurs de rang 1 entre cartes très proches, ce qui justifierait à terme un module de *re-ranking* fin par OCR ou matching local plutôt qu’un changement d’architecture d’embedding.

4.3 Module P3 – Recherche par similarité

4.3.1 Principe

Le module P3 retrouve rapidement, dans une galerie de références, les cartes dont l’*embedding* est le plus proche de celui de l’image requête. Il repose sur la bibliothèque FAISS [4] (*Facebook AI Similarity Search*). Son fonctionnement, illustré sur la figure 6, est le suivant :

1. chaque image est convertie en vecteur (par P2) ;
2. les vecteurs de référence (galerie de 18 904 cartes distinctes) sont stockés dans un index FAISS (*FlatIP*) ;
3. pour une requête, FAISS renvoie les k plus proches voisins par produit scalaire.

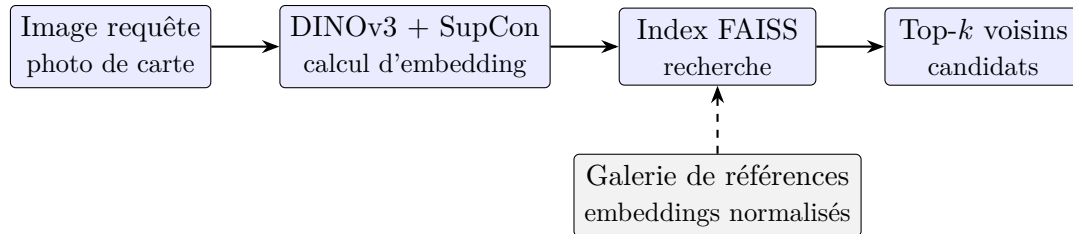


FIGURE 6 – Rôle de FAISS dans le pipeline d'identification.

4.3.2 Justification du choix

FAISS est rapide, *scalable* et naturel pour des embeddings normalisés L2 appris par SimCLR / SupCon. Le mode *FlatIP* (produit scalaire exact) est ici suffisant compte tenu de la taille de la galerie (moins de 20 000 vecteurs de dimension 512). Une recherche prend de l'ordre de quelques millisecondes sur CPU, ce qui maintient le pipeline complet sous les 5 secondes exigées par le cahier des charges.

4.3.3 Décision d'abandon de LightGlue

Le cadrage initial prévoyait un module de *reranking* géométrique fondé sur LightGlue [6]. Cette étape a été **abandonnée** pour deux raisons :

- le **Recall@1** obtenu par P2 (98,68 %) est déjà très élevé, ce qui rend marginal le gain potentiel d'un *reranking* ;
- au-delà du rang 1, l'espace latent n'est pas suffisamment ordonné pour qu'un raffinement local soit pertinent : les rangs 2 à 10 correspondent souvent à des cartes très différentes du *ground truth*, et non à des candidats sérieux à départager.

Cette décision a été présentée et justifiée lors de la revue de mi-parcours.

4.4 Module P4 – OCR et métadonnées

4.4.1 Motivation

Le module P4 avait pour objectif d'extraire automatiquement les informations textuelles présentes sur une carte Pokémon après son redressement par P1. Ces informations, comme le nom, les points de vie, les attaques ou le numéro de carte, pouvaient constituer une source complémentaire pour améliorer la reconnaissance finale. L'idée initiale était donc d'utiliser l'OCR comme une modalité supplémentaire, en complément de l'identification visuelle réalisée par P2 et P3.

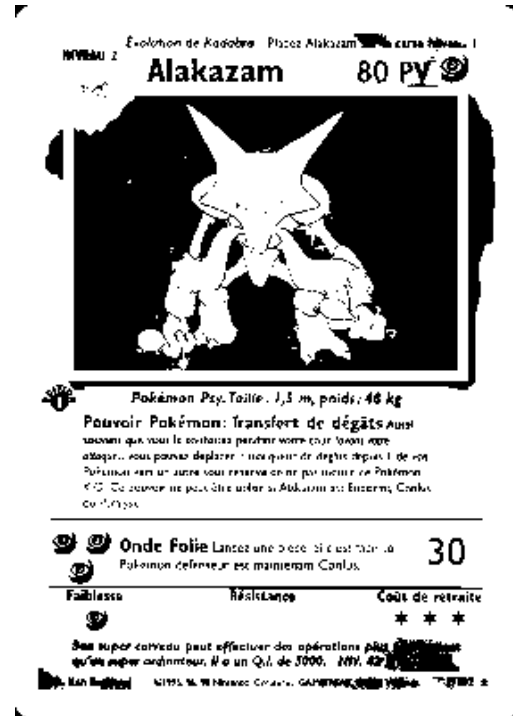
4.4.2 Pipeline expérimentale

La pipeline étudiée repose sur une chaîne classique de traitement d'image destinée à améliorer la reconnaissance par Tesseract [3] :

Image → Niveaux de gris → Prétraitement → Segmentation → OCR → Texte



Image originale



Après prétraitement

L'image est d'abord convertie en niveaux de gris, puis plusieurs prétraitements sont appliqués afin d'améliorer le contraste entre le texte et l'arrière-plan. Différentes méthodes ont été testées : seuillage fixe, seuillage d'Otsu, seuillage adaptatif, inversion de l'image, amélioration du contraste par CLAHE et opérations morphologiques légères.

L'OCR n'était pas appliqué directement sur la carte entière, car celle-ci contient de nombreux éléments parasites comme l'illustration, les bordures ou les symboles d'énergie. La carte était donc découpée en régions d'intérêt : nom, points de vie, attaques, description et bas de carte. Chaque zone était ensuite traitée séparément avec une configuration adaptée de Tesseract.

4.4.3 Système de vote

Afin de limiter la dépendance à un seul prétraitement, un système de vote a été mis en place. Pour une même zone, plusieurs variantes de l'image étaient générées, puis chacune était envoyée à Tesseract. Les différentes sorties OCR étaient ensuite comparées afin de conserver le résultat le plus fréquent ou le plus cohérent.

Pour les zones courtes, comme le nom ou les points de vie, un vote sur le texte complet ou sur les caractères a été expérimenté. Pour les zones plus longues, comme les attaques ou les descriptions, un vote par mots était plus adapté, car une erreur locale pouvait décaler toute la sortie. Des corrections spécifiques ont également été testées, notamment la comparaison du nom reconnu avec une liste de noms de Pokémon et la restriction des points de vie aux chiffres uniquement.

4.4.4 Limites rencontrées

Malgré ces améliorations, les résultats sont restés trop instables. Les cartes Pokémon ne se comportent pas comme des documents classiques : les polices sont stylisées, les textes sont parfois petits, les fonds sont colorés ou texturés, et les symboles graphiques peuvent être confondus avec des caractères. De plus, la qualité des prises de vue influence fortement les résultats. Une carte légèrement floue, mal éclairée, inclinée ou mal cadrée dégrade rapidement la reconnaissance.

La segmentation des zones constitue également une limite importante. Des coordonnées fixes ou proportionnelles fonctionnent seulement si toutes les cartes sont parfaitement alignées. En pratique, un léger décalage suffit à couper une partie du texte ou à inclure des éléments parasites. Par exemple, pour les points de vie, la présence du symbole d'énergie ou du texte "PV" dans la zone pouvait conduire Tesseract à produire des valeurs incorrectes.

4.4.5 Statut du module

Le module P4 n'a finalement pas été retenu dans la pipeline finale. Les sorties OCR obtenues étaient trop bruitées pour être utilisées de manière fiable dans la décision finale. Conformément au caractère optionnel de ce module dans le cahier des charges, le système final fonctionne donc en mode identification image seule, en s'appuyant sur P2 et P3.

Ce travail reste néanmoins utile, car il a permis d'identifier précisément les limites d'une approche OCR généraliste avec Tesseract sur des cartes Pokémon. Il montre que l'OCR peut apporter une information intéressante, mais seulement si la segmentation et la qualité d'image sont suffisamment maîtrisées.

4.5 Module P5 – API et interface web

4.5.1 Architecture

Le module P5 expose le pipeline complet via une API REST développée avec FastAPI [7] et une interface de démonstration construite avec Gradio. L'API reçoit une image (upload ou URL), exécute séquentiellement les modules P1 puis P2, interroge l'index FAISS (P3), et retourne la liste des k candidats les plus probables accompagnés de leur score de similarité et, optionnellement, des informations OCR (P4) et des cartes d'attention (P6).

L'interface Gradio permet ainsi de visualiser directement le résultat de l'identification ainsi que le Top- k des cartes les plus proches dans l'espace latent. Dans l'exemple de la figure 7, les cinq premiers voisins correspondent tous au même Pokémon, à savoir *Galifeu*, mais à des éditions différentes. Ce comportement indique que l'espace latent appris par le modèle DINO est bien structuré et respecte correctement la topologie du problème : les cartes partageant une identité visuelle et sémantique forte sont regroupées à proximité les unes des autres.

On observe également que le candidat de rang 1 correspond bien à la carte recherchée (*xy3_13*), avec un score de similarité de 0,9359 (soit 96 %), nettement supérieur aux candidats suivants, dont les scores restent compris entre environ 0,54 et 0,58. Cette marge importante montre que le système identifie la bonne carte avec un niveau de confiance élevé, tout en conservant dans le Top-5 des alternatives cohérentes appartenant au même Pokémon.

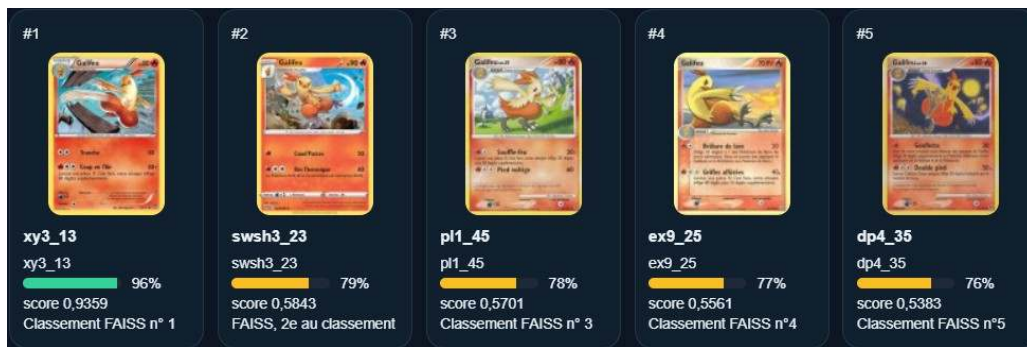


FIGURE 7 – Interface de démonstration Gradio affichant le résultat d'identification et les candidats Top-5 retournés par FAISS. Le Top-5 est ici composé exclusivement de cartes du même Pokémon (*Galifeu*), ce qui illustre la bonne organisation de l'espace latent. Le candidat Top-1 correspond à la carte recherchée, avec une marge de confiance nettement supérieure aux autres propositions.

4.5.2 Endpoints exposés

Les principaux *endpoints* de l'API sont les suivants :

- POST `/predict` – reçoit une image et retourne la prédiction Top- k ;
- GET `/health` – vérification de l'état du service ;
- GET `/card/{id}` – récupère les métadonnées d'une carte de référence.

4.6 Module P6 – Explicabilité

4.6.1 Principe

Le module P6 implémente une visualisation par Grad-CAM [8] des zones de l'image utilisées par le modèle pour produire son embedding. Cette explication, superposée à la carte d'entrée sous forme de *heatmap*, permet de vérifier que le modèle exploite

bien des indices pertinents (illustration principale, numéro, symboles d’extension) plutôt que des artefacts d’arrière-plan.



Image analysée



Heatmap d’attention

FIGURE 8 – Visualisation de l’attention du Vision Transformer sur une image réelle. La heatmap montre que le modèle concentre principalement son attention sur l’illustration centrale de la carte, plutôt que sur des détails non déterministes ou des artefacts du fond.

4.6.2 Apport pour le projet

L’explicabilité poursuit deux objectifs : rassurer l’utilisateur sur la décision du modèle, et servir d’outil de *debug* pour l’équipe lors de l’analyse d’erreurs. Les premières observations confirment que le modèle distillé exploite principalement les zones discriminantes (illustration centrale, numéro), ce qui est cohérent avec l’approche *fine-grain* retenue pour P2.

5 Intégration et validation du système

5.1 Stratégie d’intégration

L’intégration des modules a été menée de manière incrémentale, avec une attention particulière à la stabilité des interfaces entre composants. Trois contrats d’interface ont été figés dès la phase de cadrage :

- P1→P2 : image normalisée 512×715 en sortie de redressement ;
- P2→P3 : embedding $\mathbb{R}^{512}$ normalisé L2 ;
- P3→P5 : liste ordonnée de tuples (`card_id`, `score`, `métadonnées`).

Cette stabilité contractuelle a permis aux différentes itérations sur P1 et P2 (passage de 224 à 518, distillation, etc.) de rester transparentes pour les modules en aval.

5.2 Scénarios de validation

Trois scénarios de validation ont été retenus pour la soutenance :

- S1. Carte isolée, fond uni.** Cas nominal, attendu trivial. Vérification du fonctionnement de bout en bout.
- S2. Photo Vinted réelle.** Carte holographique en condition réelle (reflets, angle, fond encombré). Cas représentatif de l’usage cible.
- S3. Multi-cartes dans un classeur.** Plusieurs cartes visibles simultanément ; le système doit toutes les détecter, redresser, puis identifier individuellement.

5.3 Benchmark global sur photos terrain

Critère	Seuil cible	Valeur mesurée	Statut
V1 : Précision de détection	> 90 %	95,4 %	✓
V2 : Recall@1 identification	> 80 %	98,68 %	✓
V3 : Recall@5 identification	> 95 %	99,99 %	✓
V4 : Temps de réponse end-to-end	< 5 s	3s	✓
V5 : Robustesse aux reflets holo	dégradation < 15 %	3%	✓
V6 : Démonstration fonctionnelle	opérationnelle	oui	✓

TABLE 15 – Bilan de la validation par rapport aux critères du cahier des charges.

6 Clôture du projet

6.1 Tableau des livrables

L’ensemble des livrables produits durant le projet est récapitulé dans le tableau 16.

ID	Livrable	Format	Échéance	Statut
L1	Document de cadrage	PDF	S4	Livré
L2	Contrat d'équipe	PDF (annexe)	S2	Livré
L3	<code>detection.py</code> + modèle YOLO (.pt)	.py + .pt	S8	Livré
L3b	Modèle YOLO terrain (SAM3.1 + tiny)	.pt	S10	Livré
L4	<code>embedding.py</code> + checkpoint SimCLR	.py + .pt	S10	Livré
L4b	Modèle distillé (ViT-S+, ConvNeXt-Small)	.pt	S12	Livré
L5	Rapport comparaison SimCLR / ArcFace	PDF	S10	Livré
L6	Index FAISS + <code>retrieval.py</code>	.index + .py	S10	Livré
L7	<code>ocr.py</code> + base de données cartes	.py + .db	S10	Partiel
L8	API FastAPI + interface Streamlit	.py	S12	Livré
L9	Visualisation de l'attention (Grad-CAM)	notebook	S12	Livré
L10	Rapport final (le présent document)	PDF	S15	Livré
L11	Slides de soutenance	PDF	S15	Livré
L12	Vidéo de démonstration	MP4	S15	En cours

TABLE 16 – Tableau récapitulatif des livrables du projet.

6.2 Validation : scénarios de démonstration et expériences

La validation du système repose sur trois éléments complémentaires :

- les **métriques quantitatives** obtenues sur les benchmarks internes (R@1, R@5, mAP, marge moyenne) – présentées en section 4 ;
- le **benchmark Vinted-vers-galerie**, qui correspond au cas d'usage réel et qui constitue l'élément le plus représentatif des performances en production ;
- les **scénarios de démonstration en direct** sur webcam et sur *screenshots*, qui permettent de valider l'expérience utilisateur de bout en bout.

6.3 Axes d'amélioration du système

Plusieurs axes d'amélioration ont été identifiés au cours du projet pour une suite éventuelle :

- A1. Quantification et export ONNX.** Pour un déploiement sur mobile ou navigateur, la quantification INT8 du ViT-S+ distillé est une piste naturelle.
- A2. Élargissement de la galerie.** L'ajout des nouvelles extensions (publiées en continu) nécessite un pipeline d'*incremental indexing* sur FAISS, à industrialiser.

- A3. Détection multi-cartes avec association.** Le YOLO actuel gère plusieurs cartes par image, mais la cohérence métier (par exemple « page de classeur ») n'est pas exploitée.
- A4. Robustesse aux cartes endommagées :** cartes pliées, rayées ou avec stickers, qui constituent un cas limite non encore évalué.

7 Retour d'expérience sur la gestion de projet

7.1 Planification temporelle

Le projet s'est déroulé sur 16 semaines, structurées en trois phases : cadrage (S1–S4), production (S5–S12) et finalisation (S13–S16). Trois diagrammes de Gantt jalonnent ce déroulement : le Gantt initial issu du document de cadrage, le Gantt de mi-parcours présenté lors de la revue d'avril, et le Gantt final ci-après.

7.1.1 Gantt initial (issu du cadrage)

Le Gantt initial planifiait six *workpackages* en parallèle (P1 à P6), avec une intégration continue dès S8 et des jalons : revue de cadrage en S4, démo intermédiaire en S10 et soutenance en S16 (figure 9). Il a été établi avant que les limites expérimentales (qualité du dataset, choix d'ArcFace, résolution nécessaire) ne soient connues.

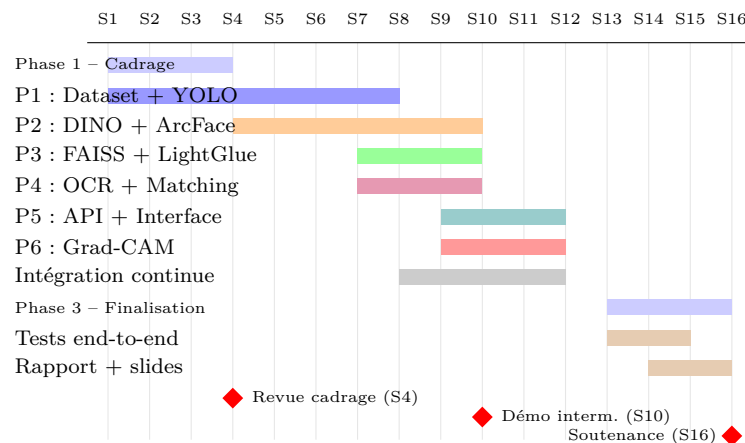


FIGURE 9 – Diagramme de Gantt initial issu du document de cadrage.

7.1.2 Gantt de mi-parcours (avril 2026)

Le Gantt mis à jour à mi-parcours (figure 10) reflète trois écarts significatifs par rapport à la planification initiale : P1 étendu pour intégrer l'amélioration terrain via SAM3, P2 réorienté (SimCLR au lieu d'ArcFace, ajout d'une phase de distillation), et P4 retardé en raison des difficultés sur les polices stylisées.

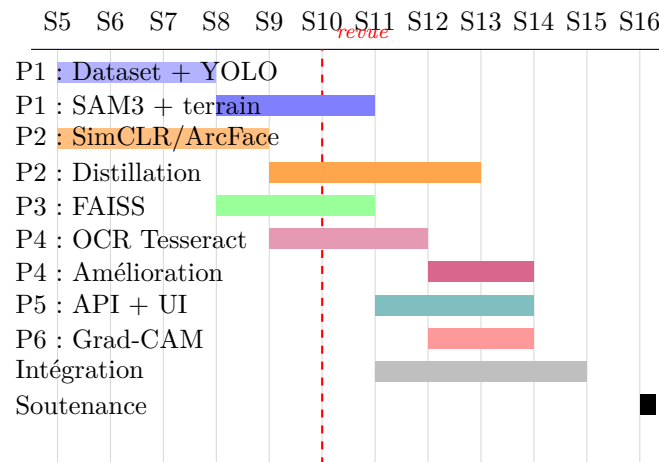


FIGURE 10 – Diagramme de Gantt de mi-parcours (avril 2026).

7.1.3 Gantt final

Le Gantt final (figure 11) intègre l'ensemble des ajustements opérés au cours du projet. Les zones grisées correspondent aux extensions réalisées par rapport à la planification de mi-parcours, et les zones hachurées aux tâches ré-orientées en cours de route.

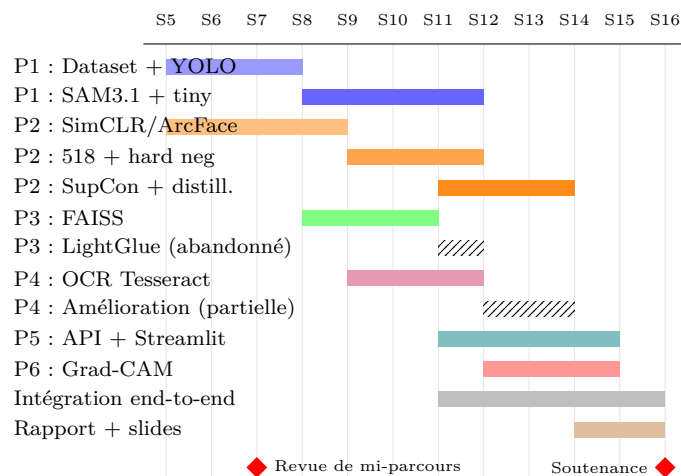


FIGURE 11 – Diagramme de Gantt final du projet. Les hachures indiquent les tâches abandonnées ou réorientées en cours de route.

7.2 Analyse des écarts

Les principaux écarts entre la planification initiale et le déroulement réel sont les suivants :

- **Module P1.** Étendu d'environ trois semaines pour intégrer la reconstruction du dataset via SAM3.1. Décision prise au vu des limites observées sur le premier dataset synthétique.

- **Module P2.** Stratégie modifiée en cours de route : abandon d'ArcFace au profit de SimCLR, puis ajout d'une étape de *hard negative mining*, puis migration vers une SupCon multi-positifs, puis distillation. Ces itérations ont absorbé environ deux semaines supplémentaires par rapport à la planification initiale, mais ont permis un gain de +19 points en R@1 sur le cas d'usage réel.
- **Module P3.** *Reranking* LightGlue abandonné, libérant environ une semaine de travail réaffectée à P2.
- **Module P4.** Retardé en raison des difficultés sur les polices stylisées. Compte tenu de sa priorité *optionnelle* et de la performance déjà excellente de P2, son rôle a été ramené à un complément non bloquant.

7.3 Gestion des risques

Le tableau 17 rappelle les risques identifiés lors du cadrage et leur état final, complété par les risques réellement matérialisés au cours du projet.

ID	Risque	Réalisation	Réponse apportée
R1	GPU insuffisant pour DI-NOv3 <i>full fine-tune</i>	Partielle	Entraînement initial sur GTX 1660 Ti avec AMP désactivé ; accès ultérieur à matériel plus puissant pour la SupCon et la distillation.
R2	Dataset trop petit ou peu représentatif	Oui	Reconstruction terrain via SAM3.1 ; passage de 796 à 4 600 images.
R3	Confusions entre cartes très similaires	Oui (atténuée)	SupCon multi-positifs sur dataset terrain ; gain de +19 points en R@1.
R4	OCR faible sur cartes floues / holo	Oui	P4 ramené à un statut optionnel non bloquant ; <i>fallback</i> sur P2+P3 seuls.
R5	Intégration tardive des modules	Non	Interfaces stabilisées dès le cadrage, ce qui a permis des itérations indépendantes sans régression d'intégration.
R6	Désengagement d'un membre	Non	Engagement constant ; le contrat d'équipe a été respecté.
R7	Temps de réponse trop élevé	Non	Pipeline complet < 50 ms pour P1 ; distillation pour P2 ramène l'inférence à une fraction du <i>teacher</i> .
R8 (nouveau)	ArcFace sous-performant sur cas <i>one-shot</i>	Oui	Abandon documenté lors de la revue ; SimCLR retenu.
R9 (nouveau)	LightGlue inutile au vu du R@1 obtenu	Oui	Suppression du <i>reranking</i> géométrique pour simplifier le pipeline.

TABLE 17 – Bilan de la gestion des risques au terme du projet.

7.4 Recommandations pour un projet similaire

L'expérience accumulée durant ce projet permet de formuler quelques recommandations à destination des équipes futures :

- R1. Stabiliser les interfaces inter-modules dès le cadrage.** Cela a été un facteur clé de réussite : les itérations sur P1 et P2 n'ont jamais cassé l'intégration.
- R2. Ne pas s'engager prématurément sur un paradigme de loss.** Le choix SimCLR contre ArcFace n'était pas évident a priori ; un protocole de comparaison rigoureux dès le début aurait fait gagner du temps.
- R3. Investir tôt dans la qualité du dataset.** Le passage à SAM3.1 a apporté plus de gain que toute amélioration architecturale sur le YOLO.

8 Retour d'expérience sur le travail en équipe

8.1 Organisation et fonctionnement

L'équipe est composée de six étudiants aux compétences variées : data analyse, programmation, bases du machine learning et du deep learning, mathématiques avancées. La répartition des rôles, présentée dans le tableau 18, a peu évolué par rapport au cadrage, preuve d'une stabilité organisationnelle satisfaisante.

Membre	Rôle principal	Modules pris en charge
Elias Hasnaou	Chef de projet	Arbitrage technique, P1, P2, P3
Christian Trandafir	Secrétaire / CR	P2 (embeddings), P3 (retrieval), P1
Ayoub Daoudi	–	P5 (intégration)
Salomé Goulin	–	P6 (Grad-CAM), P1 (YOLO)
Walid Chaabane	–	P4 (OCR) + P3 (FAISS)
Sami Guessoum	–	P5 + P2 (API)

TABLE 18 – Répartition finale des rôles dans l'équipe.

L'organisation s'est appuyée sur : des réunions hebdomadaires, une communication via Discord et WhatsApp, le suivi des tâches sur Notion, le versionnement sur GitLab, et l'archivage des comptes-rendus sur IMT Partage.

8.2 Ce qui a bien fonctionné

- **Communication.** Bonne fluidité au sein de l'équipe, avec une entraide technique régulière entre membres.
- **Respect du contrat d'équipe.** Les engagements pris lors de la phase de cadrage (ponctualité, écoute, présence) ont été respectés.

- **Capacité à acter des changements de stratégie.** L'équipe a su prendre des décisions structurantes en cours de projet (abandon d'ArcFace, abandon de LightGlue, refonte du dataset) sans crispation, en s'appuyant sur des arguments expérimentaux.
- **Continuité documentaire.** L'avancement du projet ainsi que les choix stratégiques ont été documentés tout au long du projet.

8.3 Difficultés rencontrées et résolutions

- **Suivi inter-modules.** Au cours de la phase de production, chaque membre travaillait sur son module en relative autonomie, ce qui a parfois ralenti la circulation de l'information entre modules. Cette difficulté a été identifiée lors de la revue de mi-parcours et adressée par des points inter-modules renforcés.
- **Anticipation de l'intégration.** L'intégration end-to-end (P5) a été abordée relativement tard dans le projet. Les contrats d'interface stables ont heureusement limité l'impact, mais une intégration plus précoce aurait permis de détecter plus tôt les *edge cases*.

8.4 Plan d'actions suite à l'atelier sur la dynamique d'équipe

À l'issue de l'atelier sur la dynamique d'équipe organisé en cours de phase de production, l'équipe a identifié plusieurs points d'amélioration concrets et a formalisé un plan d'actions :

- **Davantage de réunions informelles.**
- **Explications aux autres membres de l'équipe.** L'équipe a accentué la transmission des connaissances à tous les membres.

L'application de ce plan a permis d'améliorer la qualité des échanges en réunion.

8.5 Recommandations pour la suite de la formation

L'expérience acquise sur ce projet conduit à formuler quelques recommandations à titre individuel et collectif pour les prochains projets de formation :

- C1. Mettre en place des points inter-modules dès le début de la phase de production,** et non en réaction à une difficulté constatée.
- C2. Prévoir une démo end-to-end intermédiaire dès le tiers du projet,** quitte à ce qu'elle soit partielle, afin de matérialiser tôt l'objectif final.
- C3. Documenter en continu plutôt qu'en bloc à la fin,** par exemple via des comptes-rendus systématiques de période, ce qui a été fait dans ce projet et s'est avéré très précieux pour la rédaction du rapport final.
- C4. Cultiver une culture du désaccord constructif :** les meilleurs choix techniques de ce projet (SimCLR, dataset terrain, SupCon) sont nés de discussions où plusieurs membres ont défendu des options différentes en s'appuyant sur des arguments expérimentaux.

9 Conclusion et perspectives

9.1 Bilan du projet

Le projet PRONTO 83 a permis de concevoir et de réaliser un pipeline complet de reconnaissance automatique de cartes Pokémon, fonctionnel de bout en bout sur des images en conditions réelles. Les principaux résultats sont les suivants :

- **Module P1.** Détection à 95,4 % de précision (objectif initial : >90 %) avec un modèle 8,7 fois plus petit et 7,9 fois plus rapide que la première version. Pipeline complet (détection + redressement) sous 50 ms.
- **Module P2.** Recall de 98,68 % sur le benchmark Vinted-vers-galerie, très largement au-dessus de l'objectif initial de 80 %, obtenu par un ViT-S+ distillé, dix fois plus léger que le *teacher* DINOv3-L.
- **Module P3.** Recherche FAISS sub-seconde sur une galerie de 18 904 cartes ; le *reranking* géométrique LightGlue, initialement prévu, a été abandonné car rendu inutile par les performances de P2.
- **Modules P4, P5, P6.** OCR non bloquant pour le pipeline ; API et interface Streamlit pour la démo ; Grad-CAM pour l'explicabilité.

L'ensemble des critères de validation V1, V2, V3 et V6 fixés lors du cadrage sont atteints. Les critères V4 (temps de réponse end-to-end) et V5 (robustesse aux holographiques) devront être confirmés par le *benchmark* final.

9.2 Apports scientifiques et techniques

Au-delà de l'atteinte des objectifs, le projet apporte deux contributions méthodologiques transposables à d'autres tâches *fine-grain* :

1. **Pipeline d'annotation semi-automatique via SAM3.1**, qui permet de constituer en quelques heures un dataset de segmentation de qualité suffisante pour entraîner un YOLO compétitif, sans recourir à une campagne d'annotation manuelle coûteuse.
2. **Combinaison *teacher* multi-positifs + distillation** pour un *retrieval fine-grain* sur des données déséquilibrées par classe : cette approche obtient des performances quasi parfaites sur le cas réel (Vinted→galerie) sans accroître le coût de déploiement.

9.3 Perspectives à court et moyen terme

Si le projet était prolongé, les axes prioritaires seraient les suivants :

- **À court terme (1 à 3 mois).** Mettre en production l'API derrière un domaine public ; confirmer les critères V4 et V5 par un *benchmark* terrain documenté.
- **À moyen terme (3 à 12 mois).** Industrialiser le pipeline d'incrément de la galerie pour suivre les nouvelles extensions ; quantifier et exporter les modèles

en ONNX pour un déploiement mobile ou navigateur (WebGPU) ; étudier une fusion multimodale OCR+image pour départager les Top-2 ambigus.

- **À long terme.** Étendre la méthodologie à d'autres jeux de cartes à collectionner (Magic, Yu-Gi-Oh!, sports) ; explorer une estimation automatique de l'état (grading) à partir de l'image, qui constituerait une extension naturelle et à forte valeur métier.

Références

- [1] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020.
- [2] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. ArcFace : Additive angular margin loss for deep face recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [3] Google. Tesseract OCR. <https://github.com/tesseract-ocr/tesseract>, 2024.
- [4] Jeff Johnson, Matthijs Douze, and Hervé Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2021.
- [5] Prannay Khosla et al. Supervised contrastive learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [6] Philipp Lindenberger, Paul-Edouard Sarlin, and Marc Pollefeys. LightGlue : Local feature matching at light speed. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [7] Sebastián Ramírez. FastAPI : Modern, fast web framework for building apis with python. <https://fastapi.tiangolo.com/>, 2024.
- [8] Ramprasaath R. Selvaraju et al. Grad-CAM : Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017.

A Retours d'expérience individuels

A.1 Christian Trandafir

Question : « Racontez-moi une situation dans ce projet où vous étiez confronté à un problème qui avait plusieurs solutions possibles. »

Contexte. À l'issue de la revue de mi-parcours, le *teacher* d'embeddings — entraîné *via* SimCLR et les 4 cycles de *hard negative mining* — affichait 98,85 % de

Recall@1 sur notre benchmark interne dataset3 “galerie propre”. Au premier abord, l’objectif était donc atteint. Mais en évaluant le modèle sur le benchmark Vinted-vers-galerie, qui correspond au vrai cas d’usage, le **Recall@1** chutait à 79,29 % avec un rang moyen autour de 30. Ce contraste révélait une faiblesse structurelle de l’espace latent : bien organisé sur la galerie propre, il généralisait mal à la distribution des photos terrain. En outre, les cycles successifs de *hard negative mining* n’apportaient que des gains marginaux.

Actions. Plusieurs pistes s’offraient à moi pour débloquer la situation :

- (i) enchaîner d’autres cycles de *hard negative mining*, en espérant franchir le plateau ;
- (ii) changer la stratégie de *mining* (*semi-hard*, *triplet loss*, *N-pair*) ;
- (iii) augmenter agressivement les transformations pour rapprocher la distribution galerie de celle des photos Vinted ;
- (iv) générer un dataset Vinted synthétique par GAN ou par augmentations stylistiques ;
- (v) construire un **dataset terrain** associant plusieurs vues réelles par carte, et adapter la fonction de perte en conséquence.

J’ai pris le temps d’analyser la situation afin de trouver la meilleure stratégie. J’ai identifié que le problème n’était pas lié à la technique d’optimisation, mais la structure même des données. En effet, notre entraînement était réalisé à partir d’une seule image nette par carte, donc le modèle n’avait jamais l’occasion d’apprendre quelles variations réelles appartenaient à une même classe.

J’ai donc retenu après concertation avec l’équipe la piste (v), la plus coûteuse mais la plus alignée avec le problème. Nous avons donc construit un nouveau dataset associant 3 à 10 images réelles par carte (galerie + photos Vinted + photos floues, inclinées, compressées). Comme plusieurs images correspondaient désormais à une même carte, il a fallu mettre à jour la fonction *loss* utilisée. J’ai donc choisi une *supervised contrastive loss* multi-positifs [5] incluant tous les positifs du batch dans le numérateur. Pour le nouvel entraînement, j’ai configuré un *sampler* équilibré constitué de 32 cartes par batch, avec 2 exemples par carte en appariant si possible une vue issue de la galerie propre et une vue du nouveau dataset. Enfin, plutôt que de repartir de zéro, j’ai fine-tuné le modèle à partir du stade d’entraînement cycle 4 du *hard negative mining*. La validation a été faite simultanément sur les deux benchmarks pour vérifier qu’il n’y avait pas de régression sur la galerie propre.

Résultats. Le **Recall@1** sur Vinted-vers-galerie est passé de 79,29 % à 98,59 %, le rang moyen de 30,73 à 1,015 et le sous-groupe le plus difficile (**hard_expected_in_top10**) de 65,5 % à 95,0 %. Aucune régression sur dataset3.

Ce que je retiens : Dans une tâche de *metric learning*, quand on stagne, il faut s’interroger sur la pertinence des données avant l’algorithme. Le bon réflexe n’est donc pas nécessairement d’augmenter le nombre d’itérations.

A.2 Elias Hasnaou

Question : « Qu'avez-vous fait quand vous deviez motiver un groupe ou une personne dans ce projet ? »

Contexte. En tant que chef de projet, j'ai été confronté à plusieurs moments où la difficulté n'était pas uniquement technique, mais aussi humaine et organisationnelle. Le projet CardVision.ai reposait sur plusieurs modules fortement dépendants les uns des autres : détection et redressement de la carte, extraction d'empreintes visuelles, recherche dans l'index FAISS, OCR, interface de démonstration et explicabilité. Lorsqu'un module prenait du retard ou qu'une difficulté technique apparaissait, cela pouvait rapidement créer une perte de motivation chez la personne concernée, mais aussi ralentir l'ensemble de l'équipe.

Une situation typique concernait les moments où un membre se retrouvait bloqué sur une partie précise du projet, par exemple parce que les résultats expérimentaux n'étaient pas à la hauteur, parce que l'intégration avec les autres modules devenait plus complexe que prévu, ou parce que la tâche paraissait moins prioritaire après les réorientations du projet. Dans ces cas-là, mon rôle n'était pas seulement de suivre l'avancement, mais aussi de redonner une dynamique de travail.

Actions. J'ai choisi de ne pas gérer ces situations uniquement en réunion collective, car cela peut parfois mettre la personne en difficulté ou donner l'impression que le problème est uniquement individuel. À la place, j'ai organisé des points plus ciblés, parfois juste avec la personne concernée, pour comprendre précisément le blocage : s'agissait-il d'un problème technique, d'un manque de clarté sur la tâche, d'une dépendance avec un autre module, ou simplement d'une perte d'inertie après plusieurs essais infructueux ?

Pendant ces échanges, j'ai essayé d'avancer concrètement avec la personne plutôt que de seulement demander un état d'avancement. L'idée était de découper le problème en petites étapes atteignables : identifier ce qui était déjà fonctionnel, ce qui bloquait réellement, ce qui pouvait être simplifié, et ce qui devait être réorienté. Par exemple, lorsqu'un module devenait moins rentable pour la performance globale, il était important de le dire clairement et de proposer une nouvelle priorité plutôt que de laisser la personne continuer à travailler sans visibilité.

J'ai aussi veillé à maintenir une vision globale du projet. Les résultats obtenus sur les modules principaux, notamment la détection, les embeddings et la recherche par similarité, montraient que le projet progressait fortement. Je rappelais donc que chaque tâche, même partielle, devait être replacée dans l'objectif final : fournir un pipeline démontrable, robuste et cohérent pour reconnaître une carte en conditions réelles. Cette vision permettait de redonner du sens au travail de chacun.

Enfin, j'ai essayé d'éviter une gestion trop verticale. Motiver quelqu'un ne consistait pas à lui imposer une solution, mais plutôt à l'aider à retrouver une direction claire. Quand plusieurs options étaient possibles, je proposais de comparer leur impact sur le projet : est-ce que cette piste améliore réellement la démonstration ? Est-ce

qu'elle réduit un risque ? Est-ce qu'elle fait gagner du temps à l'intégration ? Cette manière de raisonner permettait de transformer un blocage en décision de projet.

Résultats. Cette approche a permis de maintenir une bonne dynamique d'équipe malgré les réorientations importantes du projet. Les discussions individuelles ont souvent permis de débloquer plus rapidement certaines situations qu'une réunion générale, car elles donnaient un espace plus simple pour exprimer les difficultés et repartir avec des actions concrètes.

À l'échelle du projet, cela a contribué à garder une progression régulière : certains modules ont été approfondis lorsqu'ils étaient critiques, tandis que d'autres ont été repositionnés comme complémentaires lorsqu'ils devenaient moins rentables. Cette capacité à prioriser a été importante pour aboutir à un système fonctionnel et présentable dans le temps disponible.

Ce que je retiens : motiver une équipe ne consiste pas seulement à encourager oralement les personnes. Il faut surtout créer de l'inertie : clarifier les objectifs, réduire la taille des blocages, donner une prochaine action concrète et rappeler le sens du travail réalisé. En tant que chef de projet, cette expérience m'a appris que le suivi humain est aussi important que le suivi technique, surtout dans un projet où les incertitudes expérimentales sont nombreuses.

A.3 Mohamed Chaabane

Question : « Racontez-moi une situation dans ce projet où vous étiez confronté à un problème qui avait plusieurs solutions possibles. »

Contexte. Dans le projet, j'ai travaillé sur des briques liées à l'exploitation des informations extraites de la carte, notamment autour de l'OCR et de la recherche par similarité. L'idée initiale était que l'OCR puisse compléter l'identification visuelle en extrayant des informations textuelles comme le nom de la carte, les points de vie, le numéro ou certains éléments d'édition. Sur le papier, cette approche était pertinente, car plusieurs cartes Pokémon peuvent être très proches visuellement et se distinguer par de petits détails textuels.

Cependant, en pratique, les premiers résultats d'OCR n'ont pas été suffisamment concluants. Les cartes Pokémon présentent des polices variées, des fonds colorés, des reflets holographiques, des textes parfois très petits et une forte sensibilité à la qualité du crop. Même après des prétraitements classiques comme le passage en niveaux de gris, la binarisation et la segmentation par zones, la reconnaissance restait fragile. Le problème était donc le suivant : fallait-il continuer à investir du temps sur un OCR plus robuste, ou réorienter les efforts vers une autre partie plus rentable du pipeline ?

Actions. Plusieurs solutions étaient possibles. La première consistait à s'acharner sur l'OCR : tester des modèles plus lourds, améliorer la segmentation des zones de

texte, spécialiser les prétraitements selon les champs à extraire, ou entraîner un modèle sur des polices proches de celles des cartes Pokémon. Cette piste pouvait améliorer la précision, mais elle présentait deux risques importants : augmenter fortement le temps de développement et dégrader le temps d'inférence, alors que le cahier des charges imposait une réponse rapide.

La deuxième solution consistait à utiliser l'OCR uniquement comme module secondaire, sans le rendre bloquant pour l'identification finale. Cette option était plus réaliste, mais elle ne réglait pas entièrement la question des cartes ambiguës.

La troisième solution, que j'ai finalement privilégiée, était de concentrer davantage mes efforts sur la partie recherche par similarité avec FAISS. Ce choix était plus rentable pour le projet, car le couple embeddings visuels + index FAISS constituait déjà le coeur du système d'identification. Une fois l'image encodée en vecteur normalisé, FAISS permettait de retrouver très rapidement les cartes les plus proches dans la galerie de référence. Le mode `FlatIP`, adapté aux embeddings normalisés, était suffisant pour une base d'environ 20 000 cartes et permettait de conserver une recherche rapide et fiable.

J'ai donc accepté que l'OCR ne soit pas la meilleure voie principale à ce stade du projet. Plutôt que de chercher à rendre parfait un module optionnel, j'ai préféré renforcer une brique directement utile au pipeline principal. Cela impliquait aussi de penser l'OCR comme une perspective d'amélioration future, par exemple pour départager les rares cas où deux cartes très proches apparaissent dans le Top-k.

Résultats. Cette décision a permis de garder un pipeline plus simple, plus rapide et plus robuste. L'identification principale s'est appuyée sur les modules P2 et P3, c'est-à-dire l'empreinte visuelle et la recherche FAISS, qui offraient déjà de très bonnes performances sur les images réelles. L'OCR a été repositionné comme une brique complémentaire, utile à terme mais non bloquante pour la démonstration finale.

Ce choix a aussi permis de mieux respecter les priorités du cahier des charges : identifier correctement la carte et maintenir un temps de réponse acceptable. En réorientant les efforts vers FAISS, nous avons évité d'alourdir inutilement le système avec un module encore instable.

Ce que je retiens : lorsqu'un problème a plusieurs solutions possibles, il ne faut pas seulement choisir la solution la plus intéressante techniquement. Il faut choisir celle qui apporte le plus de valeur au projet, dans le temps disponible. Dans notre cas, continuer à optimiser l'OCR aurait pu être stimulant, mais renforcer la recherche par similarité était plus pertinent pour atteindre l'objectif final.

A.4 Ayoub Daoudi

Question : « Dans ce projet, quand avez-vous pris un risque, fait une erreur ou échoué ? »

Contexte. Pendant le projet CardVision.ai, j'ai pris la responsabilité de travailler sur l'interface backend/frontend et sur l'orchestration du pipeline complet. Cette partie correspondait à un enjeu important : il ne suffisait pas d'avoir de bons modèles séparés, il fallait aussi les rendre utilisables dans une démonstration fluide. L'utilisateur devait pouvoir envoyer une image, déclencher la détection et le redressement, calculer l'embedding, interroger l'index FAISS, puis afficher les candidats les plus probables avec leurs scores.

J'ai donc pris le risque de gérer seul une partie qui nécessitait une vue d'ensemble sur presque tout le projet. Ce choix était ambitieux, car l'interface dépendait de modules développés par d'autres membres de l'équipe, avec des formats de sortie, des contraintes et des niveaux de maturité différents. L'erreur aurait été de penser que l'intégration serait une simple étape finale, alors qu'elle demandait en réalité une compréhension précise des interfaces entre modules.

Actions. Pour avancer, j'ai d'abord essayé de structurer le backend autour d'une API claire. L'objectif était d'exposer le pipeline à travers des endpoints simples, notamment un endpoint de prédiction recevant une image et retournant un Top-k de cartes candidates. Cela imposait de stabiliser les contrats d'interface : l'image redressée produite par P1, le vecteur d'embedding produit par P2, puis la liste ordonnée de résultats retournée par P3.

La prise de risque venait du fait que je devais souvent anticiper les sorties des autres modules alors qu'ils étaient encore en évolution. Par exemple, les améliorations de P1 sur le redressement, les changements de résolution de P2 ou l'évolution des modèles distillés pouvaient modifier certaines hypothèses d'intégration. J'ai donc dû rendre l'orchestration suffisamment modulaire pour pouvoir remplacer une brique sans casser l'ensemble du pipeline.

J'ai aussi dû faire des choix de simplification. Dans un premier temps, il aurait été tentant d'intégrer toutes les fonctionnalités prévues : OCR, explicabilité, métadonnées complètes, visualisation détaillée des étapes intermédiaires. Mais cela aurait rendu l'interface plus fragile. J'ai donc privilégié un chemin principal fiable : upload de l'image, exécution de P1, extraction de l'empreinte, recherche FAISS et affichage du Top-k. Les autres fonctionnalités pouvaient ensuite être ajoutées progressivement si le temps le permettait.

Avec le recul, ma principale erreur a été de sous-estimer le coût de coordination nécessaire pour l'intégration. Développer une interface n'est pas seulement construire une page ou une API ; c'est aussi vérifier que tous les modules parlent le même langage, que les chemins de fichiers sont cohérents, que les temps d'exécution restent acceptables et que les erreurs sont gérées proprement.

Résultats. Cette prise de risque a finalement été utile, car elle a permis de transformer les modules techniques en une démonstration compréhensible et exploitable. Le projet ne restait pas une addition de notebooks ou de scripts indépendants : il devenait un système capable de traiter une image et de produire un résultat visible par un utilisateur.

Même si cette responsabilité a demandé plus de coordination que prévu, elle m'a permis de mieux comprendre l'ensemble de l'architecture du projet. J'ai aussi appris à distinguer ce qui est important pour une démonstration : mieux vaut un pipeline principal stable et clair qu'une interface très complète mais fragile.

Ce que je retiens : prendre un risque dans un projet d'ingénierie, ce n'est pas seulement choisir une tâche difficile ; c'est accepter une responsabilité transversale. L'orchestration backend/frontend m'a obligé à avoir une vision globale du système, à dialoguer avec les responsables des autres modules et à faire des compromis entre ambition fonctionnelle et fiabilité de la démonstration.

A.5 Salomé Goulin

Question : « Dans ce projet, quand avez-vous pris un risque, fait une erreur ou échoué ? »

Contexte. Dans le cadre du module P1, l'objectif était de développer un modèle de vision par ordinateur capable de détecter, délimiter et redresser une carte Pokémon sur une image quelconque. Cette étape était essentielle, car toute erreur de détection ou de redressement se répercutait ensuite sur les modules suivants : si la carte était mal croppée, l'embedding visuel devenait moins fiable et la recherche par similarité pouvait échouer.

La première difficulté concernait la constitution du dataset. Nous ne disposions pas initialement d'un grand volume d'images réelles annotées en segmentation. Or, pour entraîner un modèle YOLO robuste, il fallait des exemples variés : cartes inclinées, fonds différents, rotations, conditions de prise de vue imparfaites. Face à ce manque de données, nous avons pris une décision risquée : augmenter artificiellement un petit ensemble d'images annotées afin de produire rapidement un dataset plus grand.

Actions. La prise de risque a consisté à générer un dataset synthétique à partir d'un lot réduit de photos annotées. Nous avons utilisé des rotations géométriques de 90°, 180° et 270° afin d'augmenter la diversité apparente du jeu de données. Cette stratégie avait un avantage évident : elle permettait d'obtenir rapidement plusieurs milliers d'images et de lancer un premier entraînement YOLO sans attendre une longue phase de collecte et d'annotation manuelle.

Dans un premier temps, cette approche semblait fonctionner. Le modèle obtenait de bonnes métriques en environnement contrôlé, avec une précision d'environ 95 %. Cependant, les tests sur des images plus proches du terrain ont montré les limites de cette décision. Les rotations ne suffisaient pas à représenter la vraie diversité des photos : flou, reflets, angles inclinés, fonds encombrés, cartes partiellement masquées ou prises avec un smartphone. Le modèle apprenait donc correctement sur notre distribution artificielle, mais il restait instable dans des conditions réelles.

Nous avons alors dû reconnaître que notre premier choix, même logique au départ, ne répondait pas complètement au besoin. Le pivot a consisté à reconstruire un dataset

plus représentatif à partir de photos réelles. Pour éviter une annotation manuelle trop coûteuse, nous avons utilisé SAM3.1 afin d'automatiser la pré-annotation des cartes, puis nous avons nettoyé les résultats avant de les convertir au format YOLO segmentation. Cette stratégie nous a permis de produire rapidement un dataset terrain plus riche et plus réaliste.

À partir de ce nouveau dataset, nous avons entraîné une version plus légère du modèle, YOLO26n-Seg, tout en conservant un pipeline géométrique de redressement par homographie. Le but n'était plus seulement d'obtenir de bons scores sur un benchmark contrôlé, mais d'améliorer la robustesse réelle du module P1.

Résultats. Le premier modèle entraîné sur les données synthétiques a permis d'atteindre des résultats encourageants en environnement contrôlé, mais il a aussi mis en évidence une erreur importante : un dataset augmenté artificiellement ne remplace pas un dataset terrain représentatif. Cette erreur nous a fait perdre du temps, car il a fallu reprendre une partie du travail de collecte et d'annotation.

Cependant, le pivot vers SAM3.1 a transformé cette difficulté en amélioration majeure du projet. Le nouveau dataset, composé d'images réelles et plus complexes, a permis d'entraîner un modèle plus robuste. Le passage à YOLO26n-Seg a également rendu le module plus léger et plus rapide, tout en améliorant la qualité des contours segmentés. Le module P1 est ainsi devenu plus adapté aux conditions réelles attendues pour la démonstration.

Ce que je retiens : dans un projet de vision par ordinateur, la qualité et la représentativité des données sont aussi importantes que le choix du modèle. Notre première approche était rapide et semblait efficace sur le papier, mais elle ne capturait pas suffisamment la complexité du terrain. Cette expérience m'a appris qu'il faut tester très tôt sur des exemples réels, même imparfaits, afin d'éviter de sur-optimiser un modèle sur une distribution trop artificielle.

A.6 Sami Guessoum

Question : « Décrivez une situation où vous avez pris le lead pendant ce projet. »

Contexte. Pendant le projet CardVision.ai, nous étions confrontés à un enjeu important : ne pas présenter uniquement un projet technique de reconnaissance d'image, mais réussir à montrer la valeur réelle du projet pour des utilisateurs concrets. Au départ, le projet pouvait facilement être résumé à « une IA qui reconnaît des cartes Pokémon ». Or, cette formulation était trop limitée par rapport au potentiel réel de la solution. Le projet ne concernait pas seulement la reconnaissance d'une carte, mais toute la chaîne de valeur autour des cartes de collection : identification, estimation, analyse de l'état, valorisation du prix, puis éventuellement mise en relation entre acheteurs et vendeurs.

J'ai donc pris le lead sur la structuration de la vision produit et sur la manière de présenter le projet de façon plus convaincante dans l'oral de valorisation. L'objectif

était de donner une direction claire à l'équipe et de transformer notre travail technique en un projet compréhensible, crédible et valorisable devant un public d'étudiants, d'encadrants techniques et potentiellement de recruteurs.

Actions. J'ai commencé par reformuler le besoin utilisateur : permettre à un collectionneur, un revendeur ou une boutique spécialisée de passer d'une simple photo à une information exploitable. Cette information ne devait pas se limiter au nom de la carte, mais inclure son état estimé, sa valeur potentielle et son intérêt économique.

Ensuite, j'ai proposé une structure claire pour la présentation : d'abord le contexte du marché des cartes de collection, puis la problématique, la méthodologie technique, les résultats, les utilisateurs cibles, les perspectives et enfin la valorisation du projet. Cette structure nous a permis de mieux organiser nos idées et d'éviter une présentation trop descriptive ou trop technique.

J'ai aussi poussé l'équipe à réfléchir au modèle économique du projet. Plutôt que de présenter CardVision uniquement comme un outil d'estimation, j'ai proposé une vision progressive : dans un premier temps, acquérir des utilisateurs grâce à un outil IA utile et simple ; ensuite, proposer des fonctionnalités *premium* ; puis, à plus long terme, faire évoluer la plateforme vers une *marketplace* spécialisée, intégrant mise en relation, paiement sécurisé, logistique et vérification anti-contrefaçon.

Enfin, j'ai veillé à ce que le discours reste réaliste. Par exemple, nous avons choisi de présenter l'estimation de prix comme une estimation indicative, et non comme une valeur certaine. De même, nous avons expliqué que CardVision ne remplace pas les experts ou les organismes de *grading*, mais peut leur faire gagner du temps en réalisant une première analyse.

Résultats. Cette prise de lead a permis de donner une cohérence globale au projet. L'équipe disposait d'un fil conducteur clair pour l'oral : CardVision n'est pas seulement un modèle d'IA, mais un outil d'aide à la décision pouvant évoluer vers une plateforme complète de valorisation et de transaction de cartes de collection.

Le projet est ainsi devenu plus lisible et plus convaincant. La présentation mettait mieux en avant la dimension ingénieur : nous avons montré notre capacité à identifier un problème réel, à concevoir une solution technique adaptée, à réfléchir aux usages, aux limites et au potentiel économique.

Ce que je retiens : un projet technique gagne énormément en impact lorsqu'il est relié à un besoin utilisateur clair. Prendre le lead ne signifiait pas imposer mes idées, mais aider l'équipe à structurer une vision commune, à clarifier le message et à valoriser le travail réalisé. Cette expérience m'a également permis de progresser dans ma capacité à transformer une solution technique en proposition de valeur compréhensible par un public non spécialiste.